

GIORNO 05

Cicli (for / while): far fare al programma la stessa cosa mille volte senza scriverla mille volte

Cinquecento messaggi

L'ultima volta Giulia ha consegnato il classificatore della posta. Funziona.

Si incolla un messaggio, il programma conta le parole spia, somma i pesi, e dice se è spam. Il responsabile lo ha provato e ha annuito.

Poi le ha fatto la domanda che aspettava da una settimana:

"Bene. Adesso passalo sulla casella. Ci sono cinquecento messaggi."

Quaranta righe che elaborano UN messaggio. Per farne cinquecento ci sono

due strade: lanciarlo cinquecento volte a mano, oppure copiare quel blocco cinquecento volte e ritrovarsi con ventimila righe. Nessuna delle due risponde.

Oggi la terza strada, che è l'unica che esista davvero: si scrive il lavoro una volta sola e si dice al programma quante volte rifarlo, o fino a quando.

Cosa saprete fare stasera

Far ripetere la stessa operazione un numero preciso di volte, con `for` e `range()`

Far ripetere finché una condizione resta vera, con `while`, e farla finire quando

l'utente scrive una parola "stabilita"

Contare quanti casi soddisfano una regola e sommare valori che arrivano uno alla volta, tenendo il conto fuori dal ciclo

Scorrere un testo carattere per carattere e ricavarne conteggi

Riconoscere un ciclo che non finisce, dire quale riga manca e fermarlo con `Ctrl+C`

Prendere un programma dei giorni scorsi che lavorava su un caso solo e farlo lavorare su una sequenza intera, con le statistiche in fondo

Il ciclo è il primo concetto veramente astratto del corso: finora ogni riga faceva una cosa, da oggi tre righe ne fanno cinquanta.

I quindici capitoli della giornata

1. Ripasso attivo: una settimana senza codice · 9. La sentinella
2. Perché serve un ciclo · 10. Uscire prima e saltare un giro
3. Ripetere un numero noto di volte: for e range · 11. for o while?
4. La variabile del ciclo · 12. Cicli annidati
5. for su stringa · 13. Il ciclo di validazione
6. Contare e accumulare · 14. Errori tipici dei cicli
7. Ripetere finché serve: while · 15. Ripetere sì, conservare no
8. Il ciclo infinito

Più due laboratori guidati scritti in diretta insieme all'aula:

05.3 — TalentHub, la media dei test di ingresso (dieci passi)

05.6 — LogiSud, il listino zona per fascia di peso (sei passi)

1. RIPASSO ATTIVO: UNA SETTIMANA SENZA CODICE

Si riparte dalle dita, non dalle diapositive

1.1 Cinque micro-esercizi al posto del riassunto

#	Cosa verifica
M1	input() restituisce sempre una stringa
M2	Le f-string sanno formattare, non solo interpolare
M3	I metodi di stringa non modificano la stringa
M4	L'ordine delle soglie in un if/elif cambia il risultato
M5	Una condizione con or scritta male è sempre vera

1.2 M1 — input restituisce una stringa: il prezzo per tre

```
# 1. Lettura da tastiera: il valore che arriva è una stringa, sempre.  
#     Nessuna conversione qui: prezzo -> "12.50" (str), non 12.50 (float).  
prezzo = input("Prezzo: ")  
  
# TRABOCCHETTO: * fra str e int non è una moltiplicazione, è una ripetizione.  
#     "12.50" * 3 -> "12.5012.5012.50" (str)  
#     12.50 * 3 -> 37.5 (float)  
totale = prezzo * 3  
print(totale)
```

L'utente digita 12.50. Prima di eseguire: cosa stampa? Scrivetelo in chat, tutti, anche a caso.

1.2 M1 — cosa stampa davvero, e la correzione

```
Prezzo: 12.50  
12.5012.5012.50
```

La correzione: la conversione esplicita al momento della lettura.

```
prezzo = float(input("Prezzo: "))  
totale = prezzo * 3  
print(f"Totale: {totale:.2f} EUR")
```

```
Prezzo: 12.50  
Totale: 37.50 EUR
```

Nessun errore, risultato assurdo. input() restituisce SEMPRE una stringa, e moltiplicare una stringa per tre la ripete tre volte.

1.3 M2 — Specificatori: larghezza, separatore, decimali

```
# Un float non ha decimali "suoi": 1234.5 vale 1234.50 e 1234.5000000.
```

```
# I decimali che si vedono li decide lo specificatore di formato, non il dato.
```

```
importo = 1234.5
```

```
# 1. .2f -> due cifre dopo il punto, arrotondate.
```

```
print(f"Totale: {importo:.2f} EUR")
```

```
# 2. >12,.2f -> tre istruzioni in una: allineamento a destra su 12 colonne,
```

```
# separatore delle migliaia, due decimali. L'ordine dentro le graffe è
```

```
# fisso: prima l'allineamento, poi la virgola, poi i decimali.
```

```
print(f"Totale: {importo:>12,.2f} EUR")
```

Tre pezzi dentro le graffe, in quest'ordine. Oggi serviranno per incolonnare le tabelle che i cicli producono: una tabella storta è la prima cosa che l'aula nota, ed è sempre uno specificatore mancante.

```
# Totale: 1,234.50 EUR
```

1.4 M3 — Immutabilità: strip() e upper() restituiscono

```
zona = "  sud  "

# TRABOCCHETTO: qui non è cambiato niente. Le stringhe sono IMMUTABILI: i
#   metodi non ritoccano l'originale, ne fabbricano una nuova e la
#   restituiscono. Senza assegnazione quella nuova viene buttata via.
#   Forma giusta: zona = zona.strip().upper()

zona.strip().upper()

print(f"[{zona}]")

# [  sud  ]
```

Non è cambiato niente. Con `zona = zona.strip().upper()` la stampa diventa `[SUD]`. È la riga che stasera mangerà mezz'ora a qualcuno dentro un ciclo, dove l'assegnazione mancante si ripete cinquecento volte.

1.5 M4 — Ordine delle soglie: il ramo irraggiungibile

```
# Costanti in cima, maiuscole: i valori di soglia si ritoccano qui, non
# dentro la logica. È la regola del Giorno 01 contro i valori magici.

SOGLIA_ALTA = 200
SOGLIA_MEDIA = 100

totale = 250

# TRABOCCHETTO: una catena if/elif si ferma al PRIMO ramo vero e non guarda
# gli altri. Chi supera 200 supera anche 100, quindi il ramo del 15% non
# verrà eseguito mai: è codice morto, e Python non segnala niente.
# Le soglie si confrontano dalla più alta alla più bassa.

if totale >= SOGLIA_MEDIA:
    print("Sconto 10%")
elif totale >= SOGLIA_ALTA:
    print("Sconto 15%")
else:
    print("Nessuno sconto")

# Sconto 10%
```

1.6 M5 — or fra due confronti: la condizione a metà

```
messaggio = "buongiorno, quando arriva l'ordine 4821?"

# TRABOCCHETTO: or non distribuisce il confronto sui due lati. Python legge
#   due operandi indipendenti: a sinistra la stringa "gratis", a destra il
#   confronto "vinci" in messaggio. Una stringa non vuota vale True, quindi
#   il lato sinistro è vero da solo e il destro non viene nemmeno valutato.
if "gratis" or "vinci" in messaggio:
    print("SPAM")
else:
    print("LEGITTIMO")
```

Il messaggio è pulito. Prima di eseguirlo, mettete un voto in chat: SPAM o LEGITTIMO?

1.6 M5 — perché stampa SPAM, e la forma corretta

SPAM

Python legge la condizione così':

```
if ("gratis") or ("vinci" in messaggio):  
    ^^^^^^^  
    una stringa non vuota vale VERO
```

VERO or qualunque cosa -> VERO

La forma corretta: ogni lato di or e' una condizione completa.

```
if "gratis" in messaggio or "vinci" in messaggio:
```

LEGITTIMO

Si scrive due volte "in messaggio" e non è ridondanza: è la sola forma che Python legge come volete voi. Oggi questa regola torna con gli interessi: dentro un ciclo, una condizione sempre vera non sbaglia un caso, ne sbaglia cinquecento di fila.

2. PERCHÉ SERVE UN CICLO: IL CODICE CHE NON SI RIPETE

Quaranta righe che servono una volta sola

2.1 Il classificatore del Giorno 04, ridotto all'osso

```
# I pesi delle parole spia e la soglia: il tariffario del Giorno 04.

PESO_GRATIS = 3

PESO_CLICCA_SUBITO = 4

SOGLIA_SPAM = 5


# 1. Un dato in ingresso. UNO: questa riga viene eseguita una volta sola,
#     e quando il programma finisce non c'è modo di chiederne un altro.
messaggio = input("Messaggio: ").strip().lower()


# 2. L'accumulatore parte da zero e raccoglie i punti degli indizi trovati.
punteggio = 0


# 3. Gli if sono separati, non una catena: gli indizi si sommano fra loro.
if "gratis" in messaggio:
    punteggio += PESO_GRATIS

if "clicca subito" in messaggio:
    punteggio += PESO_CLICCA_SUBITO
```

2.1 Un programma che elabora un dato solo: dove si ferma

```
# 4. Il punteggio è un numero, il verdetto nasce dal confronto con la soglia.
if punteggio >= SOGLIA_SPAM:
    print(f"Punteggio {punteggio}: SPAM")
else:
    print(f"Punteggio {punteggio}: LEGITTIMO")

# Messaggio: Clicca subito e ricevi un buono gratis
# Punteggio 7: SPAM
```

Funziona. Ed è finito: la riga di input() viene eseguita una volta sola, e per il secondo messaggio bisogna rilanciare il programma a mano. Su cinquecento messaggi sono cinquecento lanci.

2.2 Duplicare il blocco: i tre limiti del copia-incolla

Problema	Con il copia-incolla	Con un ciclo
Quante volte	va deciso mentre si scrive il codice	si decide mentre il programma gira
Una modifica	va ripetuta in cinquecento punti	si fa in un punto solo
Il risultato d'insieme	cinquecento variabili separate	un contatore e un totale

Sono i tre motivi per cui i cicli esistono. Analogia — il cassiere al nastro: passa un articolo, legge il prezzo, lo aggiunge al totale, poi il successivo. Fa sempre lo stesso gesto, un numero di volte che non sa quando comincia e non gli serve sapere. Il gesto ripetuto è il ciclo, il totale che cresce è l'accumulatore.

2.3 for o while: la domanda che sceglie lo strumento

So quante volte devo ripetere?	Strumento	Esempio di oggi
Sì, lo so quando parto	for	stampare i 4 promemoria della giornata
No, dipende da quello che succede	while	leggere messaggi finché l'operatore scrive fine

Prima di scrivere un ciclo si risponde a una domanda sola. Sono due strumenti, non due filosofie: in molti casi vanno bene entrambi, ma uno dei due vi fa scrivere meno righe di contabilità, e quello è quello giusto.

3. RIPETERE UN NUMERO NOTO DI VOLTE: for E range

Il primo dei due cicli: quello a cui si dice quante volte

3.1 Anatomia di un for: variabile, in, due punti, corpo

```
# range(3) non è un elenco già scritto: è un generatore di numeri, produce
# 0, 1, 2 uno alla volta. `numero` è la variabile di ciclo: a ogni giro
# Python le riassegna il valore successivo, prima di eseguire il corpo.
for numero in range(3):
    # Corpo del ciclo: rientrato di quattro spazi, eseguito una volta per
    # giro. Qui dentro `numero` vale 0, poi 1, poi 2.
    print(f"Giro numero {numero}")

# Giro numero 0
# Giro numero 1
# Giro numero 2
```

SONDAGGIO, da fare PRIMA di scoprire l'output: 1 2 3 · 0 1 2 · 0 1 2 3 · errore. Le risposte sbagliate sono questo capitolo. Tre righe di output da due righe di codice, e si legge come in italiano: per ogni numero fra quelli prodotti da range(3), esegui il blocco rientrato.

3.1 I sei pezzi di un for, e cosa succede se li sbagliate

Pezzo	Cos'è	Se lo sbagliate
for	la parola chiave, minuscola	SyntaxError
numero	il nome che date al valore del giro	niente, ma un nome brutto costa caro
in	parola obbligatoria	SyntaxError
range(3)	chi produce i valori	dipende
:	i due punti in fondo alla riga	SyntaxError: expected ':'
le righe rientrate	il corpo del ciclo	IndentationError

I due punti e l'indentazione sono gli stessi dell'if del Giorno 04. Non c'è niente di nuovo lì: se sapete scrivere un if, sapete già scrivere il guscio di un for.

3.2 range(n) e range(a, b): estremo destro escluso

```
# range(1, 4): il primo argomento è l'estremo INIZIALE, incluso; il secondo
# è l'estremo FINALE, escluso. Quindi 1, 2, 3 e non 4.
# TRABOCCHETTO: per arrivare fino a n compreso si scrive range(1, n + 1).
#   Quel "+ 1" è la fonte numero uno degli errori di conteggio.
for numero in range(1, 4):
    print(f"Promemoria {numero}")

# Promemoria 1
# Promemoria 2
# Promemoria 3
```

range(3) produce 0, 1, 2: significa "tre valori", e se il primo è zero l'ultimo è due. Il conto delle quante volte resta pulito, ed è quello che interessa quasi sempre. Quando i numeri devono essere quelli veri — il promemoria 1, il promemoria 2 — si dice anche da dove partire.

3.2 I giri di `range(a, b)` sono sempre $b - a$

Forma	Cosa produce	Quanti valori
<code>range(3)</code>	0, 1, 2	3
<code>range(1, 4)</code>	1, 2, 3	$4 - 1 = 3$
<code>range(10, 13)</code>	10, 11, 12	$13 - 10 = 3$
<code>range(5, 5)</code>	niente	$5 - 5 = 0$
<code>range(5, 2)</code>	niente	il ciclo non parte nemmeno

L'estremo destro è escluso: è la regola che fa sbagliare più conti in assoluto, e non c'è un trucco per ricordarla, c'è una formula. Se volete arrivare fino a n compreso, scrivete `range(1, n + 1)`. Quel $+ 1$ lo scriverete per il resto della vostra vita da programmatori.

3.3 range(a, b, passo): il terzo argomento

```
# Tre argomenti: parti da 5, fermati prima di 21, sali di 5 alla volta.  
# L'estremo destro resta escluso anche qui: con range(5, 20, 5) la fascia  
# dei 20 kg sparirebbe, e il listino stampato in magazzino sarebbe monco.  
for peso in range(5, 21, 5):  
    print(f"Fascia fino a {peso} kg")  
  
# Fascia fino a 5 kg  
# Fascia fino a 10 kg  
# Fascia fino a 15 kg  
# Fascia fino a 20 kg
```

Il terzo argomento è di quanto si salta. L'errore da tenere d'occhio è sempre lo stesso: l'estremo destro escluso vale anche con il passo, e la fascia più alta del listino è quella che sparisce.

3.3 Il passo negativo: si scende, e dentro c'è un if

```
# Passo negativo: si parte da 3 e si scende fino a fermarsi PRIMA dello zero,  
# quindi 3, 2, 1. Con range(3, 1, -1) il conto alla rovescia finirebbe a 2.  
for rimasti in range(3, 0, -1):  
    # Dentro il corpo si può mettere tutto quello che sapete già: qui un  
    # if/else che evita di stampare "Mancano 1 pazienti".  
    if rimasti == 1:  
        print("Manca 1 paziente")  
    else:  
        print(f"Mancano {rimasti} pazienti")  
  
# Questa riga è allineata con il for: sta FUORI dal ciclo, viene eseguita  
# una volta sola, quando i giri sono finiti.  
print("[OK] Sala d'attesa vuota")
```

3.4 Da provare subito: la tabellina del sette

```
# range(1, 6) -> 1, 2, 3, 4, 5: cinque giri, cinque righe di output.  
# Il calcolo sta dentro la f-string: numero * 7 viene eseguito a ogni giro.  
for numero in range(1, 6):  
    print(f"7 x {numero} = {numero * 7}")  
  
# 7 x 1 = 7  
# 7 x 2 = 14  
# 7 x 3 = 21  
# 7 x 4 = 28  
# 7 x 5 = 35
```

Due righe. Prima di lanciarle, dite ad alta voce quante righe di output usciranno. Poi cambiate `range(1, 6)` in `range(1, 11)` e rilanciate: avete scritto la tabellina del sette senza scrivere dieci righe. Per chi vuole andare oltre: `range(1000000)` non occupa la memoria di un milione di numeri, perché non è un elenco già pronto ma un oggetto che produce il numero successivo quando gli viene chiesto.

4. LA VARIABILE DEL CICLO

Non è speciale: è una variabile normale, riassegnata a ogni giro

4.1 Dopo il ciclo: conserva l'ultimo valore

```
# Nessuno dichiara `indice` prima: la crea il for, alla prima riassegnazione.
```

```
for indice in range(3):
```

```
    print(f"Adesso indice vale {indice}")
```

```
# TRABOCCHETTO: finito il ciclo la variabile NON sparisce, e conserva
```

```
# l'ultimo valore assegnato. Dopo range(3) vale 2, non 3: chi la rilegge
```

```
# aspettandosi il numero dei giri sbaglia di uno.
```

```
print(f"Alla fine indice vale {indice}")
```

```
# Adesso indice vale 0
```

```
# Adesso indice vale 1
```

```
# Adesso indice vale 2
```

```
# Alla fine indice vale 2
```

Comodo qualche volta, trappola spesso. La variabile fra for e in non è speciale: è una variabile normale a cui Python assegna un valore diverso a ogni giro, e dopo l'ultimo giro nessuno gliene assegna un altro.

4.2 Zero giri, nessuna assegnazione: il NameError

```
for indice in range(0):  
    print(indice)  
print(indice)
```

```
Traceback (most recent call last):  
  File "/Users/giulia/corso-python/giorno_05/vuoto.py", line 3, in <module>  
    print(indice)  
    ^^^^^  
NameError: name 'indice' is not defined
```

range(0) non produce nessun valore -> il corpo non viene eseguito mai
-> `indice` non riceve nessuna assegnazione -> alla riga 3 quel nome
non esiste.

Il messaggio è preciso: quel nome non esiste. Non è un difetto di Python, è la conseguenza del fatto che nessuno le ha mai assegnato un valore. TRABOCCHETTO: capita sul serio quando gli estremi arrivano da un calcolo, per esempio range(primo, ultimo) con primo e ultimo uguali.

4.3 Il nome della variabile: i contro il nome che dice il dato

Nome	Quando è accettabile	Alternativa che si legge
i, j	un ciclo di tre righe che conta e basta	giro, riga, colonna
x	mai, in nessun caso	dipende da cosa contiene
peso, carattere	sempre	—

Nei libri e in mezzo mondo trovate i, j, k: funziona e racconta zero. La regola è quella del Giorno 01 e non cambia: il nome dice cosa contiene. Chi legge il codice fra sei mesi siete voi.

5. for SU STRINGA: SCORRERE UN TESTO CARATTERE PER CARATTERE

L'unica collezione che avete oggi, e basta per imparare il gesto

5.1 L'iterabile che avete già: for carattere in stringa

```
zona = "SUD"
```

```
# Una stringa è un ITERABILE: un dato fatto di pezzi in fila, che il for sa  
# chiedere uno alla volta. Niente range, niente indice, niente len(): la  
# lunghezza la conosce la stringa, e il ciclo finisce quando finisce lei.
```

```
for lettera in zona:
```

```
    # A ogni giro `lettera` è una stringa di UN carattere: "S", poi "U",
```

```
    # poi "D". Non è un carattere speciale, è una stringa lunga uno.
```

```
    print(f"Lettera: {lettera}")
```

```
# Lettera: S
```

```
# Lettera: U
```

```
# Lettera: D
```

for non serve solo con range: serve con qualunque cosa abbia dei valori in fila. Liste, dizionari e tuple — cioè le cose che di solito si scorrono in un programma vero — arrivano quando impareremo a conservare i dati. Il for che scriverete su una lista è la stessa identica riga che scrivete oggi su una stringa.

5.2 Contatore su stringa: quante cifre, con .isdecimal()

```
codice_spedizione = "LS-2026-MI-0042"

# Il contatore nasce PRIMA del ciclo: deve sopravvivere a tutti i giri.
cifre = 0

for carattere in codice_spedizione:
    # .isdecimal() risponde True o False su UN carattere per volta: è il
    # filtro. I trattini e le lettere non entrano nel conteggio.
    if carattere.isdecimal():
        # += 1 legge il valore attuale, ci somma uno, lo riassegna.
        cifre += 1

# Il contatore si legge DOPO, fuori dal ciclo: dentro cambierebbe a ogni giro.
print(f"Cifre nel codice: {cifre}")

# Cifre nel codice: 8
```

5.3 if/elif/else dentro il ciclo: le quattro caselle

```
# Costante in cima: l'alfabeto di riferimento si ritocca in un punto solo.
VOCALI = "aeiou"

testo = "Referto 2026"

# Quattro contatori, uno per categoria, tutti azzerati prima del ciclo.
vocali = 0
consonanti = 0
cifre = 0
altri = 0

for carattere in testo:
    # Normalizzazione: si confronta la copia minuscola, così "R" e "r"
    # finiscono nella stessa casella. Il carattere originale resta intatto.
    minuscolo = carattere.lower()
```

5.3 Smistare ogni carattere: l'ordine dei rami

```
# TRABOCCHETTO: l'ordine dei rami è progettazione, non caso. Anche una
#   vocale è una lettera, quindi il test sulle vocali va PRIMA di
#   .isalpha(): invertiti, le vocali finirebbero fra le consonanti.
if minuscolo in VOCALI:
    vocali += 1
elif minuscolo.isalpha():
    consonanti += 1
elif carattere.isdecimal():
    cifre += 1
else:
    # Il ramo else raccoglie tutto il resto: qui dentro finisce lo spazio.
    altri += 1

print(f"Vocali: {vocali}, consonanti: {consonanti}")
print(f"Cifre: {cifre}, altri: {altri}")

# Vocali: 3, consonanti: 4
# Cifre: 4, altri: 1
```

5.3 Cosa succede dentro il ciclo, carattere per carattere

carattere	minuscolo	ramo scelto	vocali	conson.	cifre	altri
-----	-----	-----	-----	-----	-----	-----
R	r	isalpha	0	1	0	0
e	e	in VOCALI	1	1	0	0
f	f	isalpha	1	2	0	0
e	e	in VOCALI	2	2	0	0
r	r	isalpha	2	3	0	0
t	t	isalpha	2	4	0	0
o	o	in VOCALI	3	4	0	0
(spazio)	' '	else	3	4	0	1
2	2	isdecimal	3	4	1	1
0	0	isdecimal	3	4	2	1
2	2	isdecimal	3	4	3	1
6	6	isdecimal	3	4	4	1

Dodici giri, quattro contatori che crescono. Il codice non cambia mai: cambiano i valori. È esattamente la cosa che i principianti non riescono a immaginare, e l'unica che vada davvero guardata.

5.4 Accumulatore di testo: concatenare dentro il ciclo

```
nome = "Poliambulatorio Aurora"

# Accumulatore di testo: parte dalla stringa VUOTA, non da " " e non da None.
# È l'elemento neutro della concatenazione, come lo zero per la somma.
iniziali = ""

# Bandiera: dice se il prossimo carattere apre una parola. Parte a True
# perché la prima lettera del testo è sempre l'inizio di una parola.
inizio_parola = True

for carattere in nome:
    if carattere == " ":
        # Lo spazio non si copia: alza la bandiera per il giro successivo.
        inizio_parola = True
    elif inizio_parola:
        # Concatenazione: iniziali NON viene modificata sul posto, le stringhe
        # sono immutabili. Si fabbrica una stringa nuova e la si riassegna.
        iniziali = iniziali + carattere.upper()
        # TRABOCCHETTO: senza questa riga la bandiera resta alzata e il
        # programma copierebbe ogni lettera della parola, non la prima.
        inizio_parola = False

print(f"Iniziali: {iniziali}")
```

6. CONTARE E ACCUMULARE

Le variabili che vivono FUORI dal ciclo sono quelle che lo rendono un programma

6.1 I sei pattern: inizializzare, aggiornare, leggere

Pattern	Si inizializza a	Dentro il ciclo	Serve per
Contatore	0	contatore += 1	quanti casi soddisfano una regola
Accumulatore	0 oppure 0.0	totale += valore	somme, importi, pesi
Massimo	il minimo possibile	if v > massimo: massimo = v	il valore più alto visto
Minimo	il massimo possibile	if v < minimo: minimo = v	il valore più basso visto
Bandiera	False	if condizione: bandiera = True	è successo almeno una volta?
Testo	""	testo = testo + pezzo	costruire una stringa

Un ciclo che gira e non lascia traccia è inutile. Sono sei righe di tabella, e sono la sostanza di quasi ogni report d'ufficio: quante spedizioni sono partite in ritardo, quanto pesano in tutto, qual è la più pesante, se ce n'è almeno una fuori zona. Cambiano i nomi, non lo schema.

6.2 L'accumulatore: $\text{totale} = \text{totale} + \text{valore}$, e la forma $+=$

```
# L'accumulatore parte da zero UNA volta sola, prima del ciclo.

totale = 0

for numero in range(1, 6):

    # Si legge da destra: prendi il totale che c'è adesso, aggiungici il
    # numero di questo giro, rimetti il risultato dentro `totale`.
    # La forma abbreviata totale += numero fa esattamente la stessa cosa.
    totale = totale + numero

    # Stampa di traccia: serve a vedere il valore crescere giro per giro.
    # In un programma finito si toglie, in fase di studio è la cosa più utile.
    print(f"Dopo {numero}: totale = {totale}")

print(f"Somma finale: {totale}")
```

6.2 La traccia del totale, giro per giro

giro	numero	totale prima	totale + numero	totale dopo
----	-----	-----	-----	-----
1	1	0	0 + 1	1
2	2	1	1 + 2	3
3	3	3	3 + 3	6
4	4	6	6 + 4	10
5	5	10	10 + 5	15

Output del programma:

```
Dopo 1: totale = 1
Dopo 2: totale = 3
Dopo 3: totale = 6
Dopo 4: totale = 10
Dopo 5: totale = 15
Somma finale: 15
```

La colonna che conta è "totale prima": è il valore sopravvissuto al giro precedente. Se qualcuno lo azzerà, quella colonna torna a zero a ogni riga — ed è la slide dopo.

6.3 Inizializzazione dentro il ciclo: l'azzeramento

```
for numero in range(1, 6):  
    # TRABOCCHETTO: questa riga è DENTRO il ciclo, quindi viene rieseguita a  
    # ogni giro e riporta il totale a zero. Il programma non somma: ricomincia  
    # cinque volte. Quello che si vede alla fine è solo l'ultimo giro.  
    totale = 0  
    totale = totale + numero  
  
# Nessun errore, nessun avviso: solo un numero sbagliato. La domanda che lo  
# smaschera: quante volte deve essere eseguita questa riga? Se la risposta è  
# "una", quella riga sta fuori dal ciclo.  
print(f"Somma finale: {totale}")  
  
# Somma finale: 5
```

Lo fanno tutti, ogni anno, e non si nota rileggendo in fretta. Analogia — il tornello della metropolitana: conta le persone che passano e non si azzeri a ogni persona, altrimenti a fine giornata direbbe "uno". Si azzeri la mattina, una volta, prima del primo passeggero.

6.4 Contatore e accumulatore insieme: la media

```
# Due variabili distinte, due mestieri distinti: una somma i valori,
# l'altra conta le occasioni. La media è il rapporto fra le due.

somma = 0

quanti = 0

for punteggio in range(60, 100, 10):

    somma = somma + punteggio

    quanti += 1

# TRABOCCHETTO: senza questo if, un ciclo che non ha fatto nemmeno un giro
# lascia quanti a 0 e la divisione solleva ZeroDivisionError. Non è
# pignoleria: con la sentinella basta che l'operatore scriva subito "fine".
if quanti > 0:

    print(f"Somma: {somma}, candidati: {quanti}")

    # :.2f arrotonda a due decimali il risultato della divisione.
    print(f"Media: {somma / quanti:.2f}")

else:

    print("[--] nessun punteggio da mediare")

# Somma: 300, candidati: 4

# Media: 75.00
```

range(60, 100, 10) produce 60, 70, 80 e 90: quattro punteggi, somma 300, media 75. Il controllo su quanti si fa prima della divisione, sempre.

6.5 Massimo e minimo: da quale valore si inizializza

```
# Il valore di partenza del massimo è il PEGGIO possibile del dominio: sotto
# a questo non può esserci niente, quindi il primo valore vero lo supera.

PUNTEGGIO_MINIMO = 0

piu_alto = PUNTEGGIO_MINIMO

for punteggio in range(40, 100, 17):
    # Confronto e riassegnazione: il massimo si aggiorna solo se il valore
    # di questo giro è più grande di quello visto finora.
    if punteggio > piu_alto:
        piu_alto = punteggio

# TRABOCCHETTO: inizializzare piu_alto a 100 lo renderebbe insuperabile e il
# programma direbbe sempre 100. Per il MINIMO vale il contrario: si parte
# dal massimo possibile, e partire da zero è l'errore gemello.

print(f"Punteggio più alto: {piu_alto}")

# Punteggio più alto: 91
```

6.6 La bandiera booleana: trovato = False prima del ciclo

Serve quando la domanda è "è successo almeno una volta?".

Prima del ciclo:

```
trovato = False
```

Dentro il ciclo:

```
if condizione: trovato = True
```

La bandiera si alza e non si abbassa più.

Attenzione a non metterla dentro un else: con else: trovato = False

l'ultimo giro cancella tutto quello che avete visto prima.

Serve in 05.5, e ogni volta che la domanda è "ce n'è almeno uno?".

7. RIPETERE FINCHÉ SERVE:

while

Il ciclo per quando la risposta a "quante volte?" non esiste ancora

7.1 while: ripetere finché una condizione resta vera

```
# 1. Inizializzazione: la variabile della condizione deve esistere PRIMA,
#     altrimenti la riga del while solleva NameError.

colli = 3

# 2. Condizione: si valuta prima di ogni giro, compreso il primo. Finché è
#     vera si esegue il corpo; appena diventa falsa si passa oltre.

while colli > 0:

    print(f"Carico un collo, ne restano {colli - 1}")

    # 3. Aggiornamento: è la riga che avvicina la condizione al falso.

    #     Senza di lei il ciclo non finisce mai.

    colli = colli - 1

print("[OK] Furgone pieno")

# Carico un collo, ne restano 2

# Carico un collo, ne restano 1

# Carico un collo, ne restano 0
# [OK] Furgone pieno
```

while si legge finché. Analogia — le scale: for è salire dieci gradini, li avete contati e alla fine del decimo vi fermate; while è salire finché non arrivate al pianerottolo, e quanti gradini siano non lo sapete e non vi serve. Ve ne accorgete quando ci siete.

7.2 I tre obblighi di un while

Obbligo	Dove sta	Se manca
Inizializzare la variabile della condizione	PRIMA del while	NameError
Scrivere la condizione	sulla riga del while	SyntaxError
Far cambiare la variabile della condizione	DENTRO il ciclo	ciclo infinito

Nell'esempio: `colli = 3` è l'inizializzazione, `colli > 0` è la condizione, `colli = colli - 1` è l'aggiornamento. Prima di lanciare un `while`, mettete il dito sulla riga che fa cambiare la condizione. Se non la trovate, non lanciatelo.

7.3 Condizione valutata prima di ogni giro: zero giri

```
# TRABOCCHETTO: qui la condizione 0 > 0 è falsa già alla prima valutazione,  
# quindi il corpo non viene eseguito NEMMENO UNA VOLTA. Il programma non  
# è rotto e non è appeso: ha semplicemente saltato il ciclo.
```

```
colli = 0
```

```
while colli > 0:
```

```
    print("Carico un collo")
```

```
    colli = colli - 1
```

```
# È il motivo per cui un riepilogo che divide per il numero di giri va
```

```
# sempre protetto: zero giri vuol dire zero elementi raccolti.
```

```
print("[--] Niente da caricare")
```

Nessun errore, nessun output dal ciclo: è il caso che manda nel panico chi si aspetta almeno un giro, perché il programma "non fa niente" e sembra rotto. Un while non deve per forza contare all'indietro: la condizione può essere qualunque cosa che a un certo punto smette di essere vera, per esempio saldo < obiettivo — e allora il ciclo si ferma DOPO il giro che l'ha rotta.

8. IL CICLO INFINITO

Quello per errore, quello voluto, e il tasto che li ferma entrambi

8.1 Ciclo infinito per aggiornamento mancante

```
giro = 0

# TRABOCCHETTO: dei tre obblighi del paragrafo 7.2 qui ne sono rispettati
#   due. Manca l'aggiornamento: `giro` vale zero e nessuno lo cambia, quindi
#   la condizione  $0 < 5$  resta vera per sempre. Nessun errore, nessun avviso:
#   il programma fa esattamente quello che c'è scritto, all'infinito.

while giro < 5:
    print(f"Giro numero {giro}")
```

Stampa Giro numero 0 per sempre. Sul terminale vedrete righe che scorrono a una velocità che non si legge, il ventilatore che parte e la finestra che non risponde più ai tasti. Analogia — il disco rotto: la puntina salta sempre allo stesso solco perché niente cambia. Per fermarla si alza la puntina.

8.2 Interrompere l'esecuzione: Ctrl+C e KeyboardInterrupt

```
Giro numero 0
Giro numero 0
Giro numero 0
^CTraceback (most recent call last):
  File "/Users/giulia/corso-python/giorno_05/infinito.py", line 4, in <module>
    print(f"Giro numero {giro}")
KeyboardInterrupt
```

Su macOS e' Ctrl+C, non Cmd+C: una delle pochissime scorciatoie che usa Ctrl.

Chiudere la finestra	-> tre minuti persi
Riavviare l'editor	-> otto minuti persi
Scollegarsi da Webex	-> quindici minuti persi
Ctrl+C	-> mezzo secondo

Questa cosa si impara adesso, prima di scrivere il vostro primo while. KeyboardInterrupt non è un errore del vostro codice: è Python che vi dice "mi hai interrotto tu", e il ^C è il vostro tasto, stampato dal terminale. Da fare adesso, tutti, sulla propria macchina, e scrivere fermato in chat.

8.3 Le quattro cause, e il sintomo di ciascuna

Causa	Sintomo	Riga da guardare
Nessuno aggiorna la variabile della condizione	la stessa riga stampata all'infinito	dentro il ciclo, cerca l'aggiornamento
L'aggiornamento è fuori dal ciclo (indentazione)	idem	il margine sinistro dell'ultima riga del corpo
L'aggiornamento va nella direzione sbagliata	numeri che crescono invece di scendere	il segno di += o -=
La sentinella non viene mai riconosciuta	il programma continua a chiedere	come normalizzi l'input prima di confrontarlo

Si parte dal sintomo, non dal codice. Il secondo caso è il più insidioso, perché il codice sembra giusto: è la slide dopo.

8.3 Il caso insidioso: quattro spazi di differenza

```
colli = 3

while colli > 0:
    print(f"Restano {colli} colli")
# TRABOCCHETTO: questa riga è allineata con il while, quindi è FUORI dal
# ciclo. Verrà eseguita quando il ciclo sarà finito, cioè mai. In Python
# l'indentazione non è estetica: sposta la riga da un blocco all'altro.
# La correzione è di quattro spazi.
colli = colli - 1
```

La correzione è di quattro spazi: si rientra la riga, e il programma stampa tre righe e si ferma. Nessun messaggio, nessun avviso: il codice è sintatticamente perfetto e semanticamente un altro programma.

8.4 Il ciclo infinito voluto: condizione sempre vera più break

Esiste anche il ciclo infinito scritto apposta, e non è un errore:

```
while True:  
    ...  
    if risposta.lower() == SENTINELLA:  
        break
```

La condizione è sempre vera: il ciclo non finisce da solo.

L'uscita la decide un break dentro il corpo.

Lo vediamo al capitolo 10, ed è la forma standard della sentinella.

La differenza con l'errore è che lì l'uscita c'è ed è scritta:

si vede a colpo d'occhio, leggendo il corpo del ciclo.

9. LA SENTINELLA

La parola convenuta che chiude il ciclo, vecchia quanto la programmazione

9.2 Forma base: la lettura che viene prima del ciclo

```
# La parola convenuta sta in una costante: si confronta in un punto solo e
# si cambia in un punto solo.

SENTINELLA = "fine"

# Accumulatore float (0.0, non 0) e contatore: il totale somma pesi con la
# virgola, e partire da un int funzionerebbe lo stesso ma dichiara il tipo
# sbagliato a chi legge.

totale = 0.0

quanti = 0

# 1. Prima lettura, FUORI dal ciclo: la condizione del while ha bisogno di
# qualcosa da controllare già al primo giro.
```

Tre cose prima ancora del while: la costante, le due variabili che devono sopravvivere a tutti i giri, e la prima lettura. Se la prima lettura manca, la riga del while solleva `NameError` sul nome `risposta`.

9.2 Forma base: la seconda lettura, in coda al corpo

```
# 2. Si resta dentro finché la risposta non è la sentinella. Il confronto
#   lavora sulla copia minuscola: "Fine" e "fine" devono chiudere entrambe.
while risposta.lower() != SENTINELLA:

    # float() converte la stringa in numero. Qui è già certo che non sia la
    # sentinella: il controllo è avvenuto sulla riga del while.

    totale = totale + float(risposta)

    quanti += 1

    # TRABOCCHETTO: questa seconda lettura è l'aggiornamento della condizione.
    #   Toglietela e il ciclo ripete all'infinito la prima risposta.

    risposta = input("Peso del collo in kg (o 'fine'): ").strip()

print(f"Colli: {quanti}, peso totale: {totale:.2f} kg")

# Peso del collo in kg (o 'fine'): 12.5
# Peso del collo in kg (o 'fine'): 8
# Peso del collo in kg (o 'fine'): 10
# Colli: 3, peso totale: 30.50 kg
```

Guardate la struttura: si legge una volta prima del ciclo, e di nuovo in fondo al corpo. Quella riga scritta due volte dà fastidio a tutti, ed è il motivo per cui al capitolo 10 impareremo la forma con `break`.

9.3 Come si sceglie la sentinella

Sentinella	Va bene?	Perché
fine	sì	si scrive in fretta, non è un dato valido
-1	dipende	va bene per un peso, non per una temperatura
0	quasi mai	zero è un valore legittimo quasi ovunque
riga vuota	con cautela	basta l'Invio, ma un invio distratto chiude tutto

La regola è una sola: la sentinella non deve poter essere un dato vero. Il giorno in cui lo diventa — una temperatura di -1, una giacenza a zero — il programma chiude il ciclo su un dato buono e nessuno se ne accorge.

9.4 Normalizzare prima di confrontare: `.strip()` e `.lower()`

```
Punteggio (o 'fine'): 72
Registrato: 72
Punteggio (o 'fine'): Fine
Punteggio (o 'fine'): FINE
Punteggio (o 'fine'): fine
Punteggio (o 'fine'):
```

```
"Fine" != "fine"    ->   VERO    ->   il ciclo continua
```

La correzione, due metodi con due mestieri diversi:

```
risposta = input("Punteggio (o 'fine'): ").strip()
while risposta.lower() != SENTINELLA:
    print(f"Registrato: {risposta}")
    risposta = input("Punteggio (o 'fine'): ").strip()
```

```
Punteggio (o 'fine'): Fine
[OK] Chiuso
```

Causa numero uno dei cicli che non si chiudono in aula. `.strip()` ripulisce il dato, `.lower()` uniforma il confronto. TRABOCCHETTO: si stampa risposta, non la copia minuscola — la normalizzazione serve per CONFRONTARE, non per rovinare il dato.

9.5 Sentinella prima della conversione: il ValueError

```
risposta = input("Punteggio (o 'fine'): ").strip()
punteggio = int(risposta)

# TRABOCCHETTO: la riga 2 converte PRIMA di ogni controllo. Finché l'utente
#   digita cifre va tutto bene; il giorno in cui digita la parola convenuta,
#   int() solleva ValueError e il programma muore lì.

while punteggio != -1:
    print(f"Registrato: {punteggio}")
    risposta = input("Punteggio (o 'fine'): ").strip()
    punteggio = int(risposta)

# Punteggio (o 'fine'): fine
# Traceback (most recent call last):
#   File "media.py", line 2, in <module>
#     punteggio = int(risposta)
# ValueError: invalid literal for int() with base 10: 'fine'
```

L'ordine giusto è: leggo, controllo se è la sentinella, e SOLO DOPO converto. Il controllo si fa sulla stringa; la conversione arriva quando si è già deciso che quella stringa è un numero.

LAB GUIDATO 1 — passo 0 e 1: il file vuoto, la domanda

```
"""Esercizio 05.3 - TalentHub, media dei test di ingresso."""
```

```
LARGHEZZA = 60
```

```
SENTINELLA = "fine"
```

```
PUNTEGGIO_MINIMO = 0
```

```
PUNTEGGIO_MASSIMO = 100
```

```
SOGLIA_IDONEITA = 60
```

```
def main():
```

```
    print("=" * LARGHEZZA)
```

```
    print("TALENTHUB - Media dei test di ingresso")
```

```
    print("=" * LARGHEZZA)
```

```
if __name__ == "__main__":
```

```
    main()
```

LAB GUIDATO 1 — passo 2: cosa esiste prima del ciclo

```
somma = 0
```

```
candidati = 0
```

```
idonei = 0
```

```
piu_alto = PUNTEGGIO_MINIMO
```

```
piu_basso = PUNTEGGIO_MASSIMO
```

```
# DA PROVOCARE ADESSO: qualcuno proporra' sempre
```

```
#
```

```
#     piu_basso = 0
```

```
#
```

```
# Scrivetelo, eseguitelo alla fine con tre punteggi veri, e fate
```

```
# vedere che il minimo resta zero per sempre, senza nessun errore.
```

```
# E' il momento migliore del laboratorio.
```

LAB GUIDATO 1 — passo 3 e 4: il guscio, la lettura, l'uscita

```
while True:
    risposta = input(f"Punteggio (o '{SENTINELLA}') : ").strip()

    if risposta.lower() == SENTINELLA:
        break
```

LAB GUIDATO 1 — passo 4: l'errore da provocare, e il Ctrl+C

```
Punteggio (o 'fine'): 72
Punteggio (o 'fine'): Fine
Punteggio (o 'fine'): Fine
Punteggio (o 'fine'): Fine
Punteggio (o 'fine'): ^CTraceback (most recent call last):
  File "/Users/giulia/corso-python/giorno_05/media.py", line 25, in main
    risposta = input(f"Punteggio (o '{SENTINELLA}')"): ").strip()
KeyboardInterrupt
```

STOP D'AULA - la frase esatta:

```
"Il programma non e' rotto e non si chiude la finestra:
si preme Control e C insieme, adesso, tutti."
```

ALLINEAMENTO:

```
"Chi non ha esattamente queste otto righe scriva stop in chat."
Aspettare davvero. Contare gli stop.
```

Si toglie .lower() dal confronto e si esegue scrivendo Fine con la maiuscola. Il ciclo non finisce: è qui che si insegna Ctrl+C, e va fatto fare a tutti, non solo mostrato.

LAB GUIDATO 1 — passo 5: validare prima di convertire

```
# Prima si scrive QUESTA, si esegue, e si legge l'errore insieme:
#
#     punteggio = int(risposta)
#
# con l'utente che scrive: settanta
#
# ValueError: invalid literal for int() with base 10: 'settanta'
```

```
if not risposta.isdecimal():
    print(" [ERRORE] scrivi solo cifre, oppure 'fine'")
    continue
```

```
punteggio = int(risposta)
if punteggio > PUNTEGGIO_MASSIMO:
    print(f" [ERRORE] il massimo è {PUNTEGGIO_MASSIMO}")
    continue
```

LAB GUIDATO 1 — passo 6: accumulare, con l'errore provocato

```
somma += punteggio  
  
candidati += 1  
  
if punteggio >= SOGLIA_IDONEITA:  
    idonei += 1  
  
print(f" [OK] registrato. Candidati finora: {candidati}")
```

```
# Prima si propone somma = 0 DENTRO il ciclo e si esegue con 72, 58, 91:  
  
#     media 91.00    -> sbagliata, e nessun errore  
  
#  
  
# Poi si sposta fuori e si riesegue con gli STESSI tre numeri:  
  
#     media 73.67    -> giusta
```

LAB GUIDATO 1 — passo 7 e 8: gli estremi, la divisione

```
if punteggio > piu_alto:
    piu_alto = punteggio
if punteggio < piu_basso:
    piu_basso = punteggio
```

```
# Eseguire e scrivere subito 'fine':
#     ZeroDivisionError: division by zero
#
# La correzione, attorno a TUTTO il riepilogo:
```

```
if candidati > 0:
    media = somma / candidati
    print(f"Media ..... {media:.2f}")
else:
    print("[--] Nessun punteggio inserito: niente da calcolare.")
```

LAB GUIDATO 1 — passo 9 e 10: il file gira, 72 58 91 64

```
=====
TALENTHUB - Media dei test di ingresso
=====
Punteggio (o 'fine'): 72
  [OK] registrato. Candidati finora: 1
Punteggio (o 'fine'): 58
  [OK] registrato. Candidati finora: 2
Punteggio (o 'fine'): novanta
  [ERRORE] scrivi solo cifre, oppure 'fine'
Punteggio (o 'fine'): 91
  [OK] registrato. Candidati finora: 3
Punteggio (o 'fine'): 64
  [OK] registrato. Candidati finora: 4
Punteggio (o 'fine'): fine
-----
Candidati registrati .. 4
Somma dei punteggi .... 285
Media ..... 71.25
Punteggio più alto .... 91
Punteggio più basso ... 58
Idonei (>= 60) ..... 3
```

10. USCIRE PRIMA E SALTARE UN GIRO: break E continue

Due parole chiave, due mestieri diversi

10.1 break: uscire dal ciclo sul posto

```
SENTINELLA = "fine"

totale = 0.0
quanti = 0

# Condizione sempre vera: il ciclo non finisce da solo, e non è un errore
# perché l'uscita c'è ed è scritta dentro il corpo, tre righe più sotto.
while True:

    # Unica lettura, dentro il ciclo: la riga duplicata del capitolo 9 sparisce.
    risposta = input("Peso del collo in kg (o 'fine'): ").strip()

    # L'uscita: break interrompe subito e salta alla prima riga fuori dal
    # ciclo, cioè alla stampa finale. Le righe sotto non vengono eseguite.
    if risposta.lower() == SENTINELLA:
        break

    # TRABOCCHETTO: questa conversione è protetta dal break che la precede.
    # Se il controllo della sentinella finisse dopo, la parola convenuta
    # arriverebbe dentro float() e il programma morirebbe con ValueError.
    totale = totale + float(risposta)

    quanti += 1

print(f"Colli: {quanti}, peso totale: {totale:.2f} kg")
```

Stesso risultato del capitolo 9 — Colli: 2, peso totale: 20.50 kg — con una riga di input() invece di due. È la forma che useremo in 05.7 e in 05.8.

10.2 continue: saltare il resto del corpo

```
PESO_MASSIMO = 30.0

accettati = 0
scartati = 0

for peso in range(10, 51, 10):
    if peso > PESO_MASSIMO:
        print(f"  [{!}] {peso} kg: sopra il limite, collo scartato")
        scartati += 1
        # continue chiude QUESTO giro e passa al successivo: le due righe
        # sotto non vengono eseguite per questo peso. Il ciclo non si ferma.
        continue

    # Si arriva qui solo se il continue non è scattato: è il caso buono.
    # TRABOCCHETTO: senza il continue servirebbe un else e un rientro in più.
    # Con il continue il caso buono resta a sinistra, e si legge meglio.
    print(f"  [OK] {peso} kg: collo accettato")
    accettati += 1

print(f"Accettati {accettati}, scartati {scartati}")
# Accettati 3, scartati 2
```

continue non esce: salta il resto del corpo e va al giro successivo. Serve soprattutto per scartare gli input sporchi in cima al ciclo — "se questa riga non è valida, dillo e passa alla prossima". In 05.3 e 05.8.

10.3 Abuso di break e continue: i due sintomi

Sintomo	Cosa c'è dietro	Cosa fare
Tre break nello stesso ciclo	la condizione di uscita è scritta male	riscrivere la condizione del while
continue come ultima riga del corpo	non serve a niente	cancellarlo

Un break e un continue per ciclo sono la norma. Se ne servono di più, quasi sempre il problema non è il break: è la condizione. Ultima cosa, che serve al capitolo 12: break esce da un ciclo solo. In un ciclo annidato chiude quello interno e lascia girare l'esterno.

11. for O while?

La decisione si prende in tre secondi, e quasi sempre è for

11.1 Numero di giri noto o ignoto: la regola di scelta

Situazione	Strumento	Perché
Stampare i 12 mesi	for	dodici, sempre
Scorrere i caratteri di un codice	for	quanti sono lo dice la stringa
Ripetere finché l'utente scrive fine	while	dipende da lui
Chiedere finché il dato non è valido	while	dipende da quanto sbaglia
Raddoppiare finché non supero mille	while	il numero di giri è il risultato, non il dato
Fare esattamente tre tentativi	for	tre
Al massimo tre, fermandosi prima se va bene	for con break	va bene anche while con due condizioni

Se sapete quante volte, for. Se dipende da quello che succede, while.

11.2 Lo stesso ciclo nei due modi: le tre righe in più

Versione for: la variabile nasce, cambia e finisce dentro la riga del for.

Una riga di contabilità, zero occasioni di sbagliare.

```
for numero in range(1, 4):
```

```
    print(f"Promemoria {numero}")
```

Versione while: la stessa cosa a mano. Tre righe di contabilità –

creazione, condizione, incremento – e tre punti in cui sbagliare.

```
numero = 1
```

```
while numero <= 3:
```

```
    print(f"Promemoria {numero}")
```

```
    # TRABOCCHETTO: togliete questa riga e il ciclo non finisce più. Nel for
```

```
    # la riga corrispondente non esiste, quindi non si può dimenticare.
```

```
    numero += 1
```

12. CICLI ANNIDATI: UN CICLO DENTRO L'ALTRO

Due livelli, non tre: i giri si moltiplicano

12.1 Ciclo esterno e interno: i giri si moltiplicano

```
# Ciclo ESTERNO: tre giri, uno per lettera della stringa "NCS".
for zona in "NCS":
    # Ciclo INTERNO: è il corpo di quello esterno, quindi riparte da capo a
    # ogni giro. Tre giri per ogni zona, non tre giri in tutto.
    for peso in range(5, 16, 5):
        # Il corpo più interno viene eseguito  $3 \times 3 = 9$  volte. I giri si
        # MOLTIPLICANO: è il conto che vi salva dal lanciare per sbaglio un
        # programma che stampa duecentomila righe.
        print(f"Zona {zona}, fino a {peso} kg")

# Zona N, fino a 5 kg      Zona C, fino a 5 kg      Zona S, fino a 5 kg
# Zona N, fino a 10 kg     Zona C, fino a 10 kg     Zona S, fino a 10 kg
# Zona N, fino a 15 kg     Zona C, fino a 15 kg     Zona S, fino a 15 kg
```

12.2 Griglia con end="": dove sta il print() a capo

```
# Il contatore sta fuori da ENTRAMBI i cicli: deve sopravvivere a tutti i
# giri di tutti e due i livelli.
celle = 0

for riga in range(1, 4):
    for colonna in range(1, 4):
        # end="" sostituisce l'a-capo automatico di print con niente: i
        # numeri di una stessa riga restano in fila. :>4 li allinea a destra
        # su quattro colonne, ed è la formattazione del Giorno 02.
        print(f"{riga * colonna:>4}", end="")
        celle += 1

    # TRABOCCHETTO: questo print() vuoto è rientrato di quattro spazi, quindi
    # sta nel corpo dell'ESTERNO e fuori dall'interno: chiude una riga
    # della griglia. Spostatelo dentro l'interno e uscirebbero nove righe
    # da un numero ciascuna.

    print()

print(f"Celle calcolate: {celle}")

# 1  2  3
# 2  4  6
# 3  6  9
# Celle calcolate: 9
```

12.3 Variabili dei due livelli: nomi diversi, sempre

Se scrivete `for numero in range(1, 3):` e dentro un altro

`for numero in range(1, 3):`, il ciclo interno sovrascrive la variabile dell'esterno a ogni giro.

Python non protesta.

Non è un errore di sintassi: è un errore di progetto.

In un programma vero produce numeri sbagliati e nessun messaggio, che è la categoria di guasto più costosa da trovare.

Nomi diversi ai due livelli, sempre:

riga e colonna · zona e peso · messaggio e parola

12.4 Da provare subito: sei etichette su due righe

```
# Due giri esterni per tre interni: sei etichette stampate in tutto.
for iniziale in "NS":
    for peso in range(10, 31, 10):
        # end=" " separa con uno spazio invece di andare a capo.
        print(f"{iniziale}{peso}", end=" ")

    # L'a-capo è qui, nel corpo dell'esterno: due righe, non sei.
    print()

# N10 N20 N30
# S10 S20 S30
```

Quattro righe. Prima di lanciarle, scrivete in chat quante righe di output prevedete. Due giri esterni, tre interni: sei etichette su due righe. Se avete risposto "sei righe", avete dimenticato che il print() vuoto è dentro l'esterno.

LAB GUIDATO 2 — passo 1: il ciclo esterno, senza liste

```
SIGLE_ZONE = "NCSI"
```

```
for sigla in SIGLE_ZONE:
```

```
    print(sigla)
```

```
# N
```

```
# C
```

```
# S
```

```
# I
```

DOMANDA: le zone sono NORD, CENTRO, SUD, ISOLE. Come le scorro, se le liste non ce le ho? Aspettare. La risposta che arriva quasi sempre è "con quattro if": accettarla, poi proporre le iniziali. "Una stringa è l'unica cosa che oggi possiamo scorrere. Più avanti questa riga diventerà una lista e sarà più leggibile: oggi funziona così, e funziona."

LAB GUIDATO 2 — passo 2: dalla sigla ai dati

```
for sigla in SIGLE_ZONE:

    if sigla == "N":

        nome_zona = "NORD"

        base = BASE_NORD

        tariffa = TARIFFA_NORD

    elif sigla == "C":

        nome_zona = "CENTRO"

        base = BASE_CENTRO

        tariffa = TARIFFA_CENTRO

    elif sigla == "S":

        nome_zona = "SUD"

        base = BASE_SUD

        tariffa = TARIFFA_SUD

    else:

        # L'else finale raccoglie l'ultimo caso: senza, una sigla non
        # prevista lascerebbe nome_zona con il valore del giro precedente.

        nome_zona = "ISOLE"

        base = BASE_ISOLE

        tariffa = TARIFFA_ISOLE
```

DOMANDA: perché non scrivo i nomi completi nella stringa da scorrere? **Risposta:** scorrendo "NORDCENTROSUD" si ottengono le singole lettere, non le parole — mostrarlo eseguendolo, due righe e tre secondi. **ALLINEAMENTO:** "chi non vede quattro nomi di zona stampati scriva stop".

```
print(f"{nome_zona:<8}", end="")
```

LAB GUIDATO 2 — passo 3: il ciclo interno e il contatore

```
# celle = 0 sta FUORI da entrambi i cicli

for peso in range(PESO_MINIMO, PESO_MASSIMO + 1, PASSO_PESO):
    # PESO_MASSIMO + 1 perché l'estremo destro è escluso: senza il + 1
    # la colonna dei 20 kg sparirebbe dal listino appeso in magazzino.
    costo = base + tariffa * peso
    print(costo, end=" ")
    celle += 1

print()
```

DOMANDA: quante volte gira in tutto il print che sta dentro? Attesa: sedici. Farlo verificare con il contatore, che poi resta nel programma finito. Nome diverso ai due livelli: sigla fuori, peso dentro. Con lo stesso nome non c'è nessun errore, solo risultati incomprensibili.

LAB GUIDATO 2 — passo 4: l'errore da provocare

Con il `print()` DENTRO il ciclo interno:

```
8.50
12.50
16.50
20.50
9.50
... sedici righe da un numero ciascuna
```

Con il `print()` spostato di quattro spazi a SINISTRA:

```
8.50      12.50      16.50      20.50
9.50      14.00      18.50      23.00
11.50     17.00      22.50      28.00
16.50     24.00      31.50      39.00
```

Non spiegarlo prima: farlo vedere, e poi chiedere all'aula di spiegarlo a voi. È la riga più importante del programma: sta DOPO il ciclo interno e DENTRO quello esterno, e manda a capo una volta per zona.

LAB GUIDATO 2 — passo 5 e 6: allineamento e cella più cara

```
celle = 0

costo_massimo = 0.0

zona_massima = ""

peso_caro = 0

for sigla in SIGLE_ZONE:
    ...
    for peso in range(PESO_MINIMO, PESO_MASSIMO + 1, PASSO_PESO):
        costo = base + tariffa * peso
        print(f"{costo:>9.2f}", end="")
        celle += 1

        # TRABOCCHETTO: le quattro variabili del riepilogo stanno fuori da
        # ENTRAMBI i cicli. Dentro l'esterno, il massimo si azzererebbe a
        # ogni zona e il programma direbbe sempre l'ultima.

        if costo > costo_massimo:
            costo_massimo = costo

            zona_massima = nome_zona
            peso_caro = peso
        print()
```

Il :>9.2f è il formato del Giorno 02: provare senza >9 e mostrare la tabella storta è l'argomento più convincente a favore della formattazione. DOMANDA sulle quattro variabili: dove le inizializzo? Se qualcuno risponde "dentro il ciclo esterno", scriverlo così ed eseguire.

LAB GUIDATO 2 — chiusura: il listino finito

```
=====
LOGISUD - Listino: zona per fascia di peso (EUR)
=====
```

ZONA	5 kg	10 kg	15 kg	20 kg
NORD	8.50	12.50	16.50	20.50
CENTRO	9.50	14.00	18.50	23.00
SUD	11.50	17.00	22.50	28.00
ISOLE	16.50	24.00	31.50	39.00

```
-----
Celle calcolate ..... 16
```

```
Cella più cara ..... ISOLE a 20 kg
```

```
Prezzo più alto ..... 39.00 EUR
=====
```

File in chat. Tutti eseguono e scrivono in chat il prezzo della cella più cara: 39.00. Poi l'aula prosegue in autonomia su 05.5; chi non finisce lo riprende come compito.

13. IL CICLO DI VALIDAZIONE: CHIEDERE FINCHÉ IL DATO NON È BUONO

Il pezzo che mancava nei Giorni 03 e 04

13.1 Dal controllo singolo alla richiesta ripetuta

```
LUNGHEZZA_CAP = 5

# Prima lettura fuori dal ciclo: la condizione ha bisogno di un dato da
# guardare già al primo giro. È la stessa struttura della sentinella.
cap = input("CAP: ").strip()

# TRABOCCHETTO: la condizione del while è la NEGAZIONE della regola di
# validità. La regola è "cinque caratteri E tutte cifre"; per restare
# dentro finché il dato è sbagliato serve "lunghezza diversa da cinque
# OPPURE non tutte cifre". Chi la scrive al dritto ottiene un ciclo che
# non parte mai, o che non finisce mai.
while len(cap) != LUNGHEZZA_CAP or not cap.isdecimal():
    print("[ERRORE] Il CAP è fatto di 5 cifre, niente lettere e niente spazi")
    # Senza questa seconda lettura il dato resta quello sbagliato e il
    # messaggio scorre all'infinito.
    cap = input("CAP: ").strip()

print(f"[OK] CAP registrato: {cap}")
```

13.2 Messaggi distinti, e un tetto ai tentativi

```
MAX_TENTATIVI = 3

LUNGHEZZA_CAP = 5

PREFIXO_PROVINCIA = "84"

# Due variabili di controllo: la bandiera dice SE si è arrivati a un dato
# buono, il contatore dice QUANTE occasioni restano. Servono entrambe.

valido = False

tentativi = 0

# Due condizioni unite da and: si continua solo se il dato non è ancora
# valido E ci sono ancora tentativi disponibili. Basta che una cada e si esce.
while not valido and tentativi < MAX_TENTATIVI:
    tentativi += 1

    cap = input(f"CAP (tentativo {tentativi} di {MAX_TENTATIVI}): ").strip()
```

13.2 L'ordine dei controlli, e la bandiera che serve dopo

```
# TRABOCCHETTO: l'ordine dei controlli è progettazione, non caso. Prima
#   la lunghezza, poi il contenuto, poi la regola di dominio: invertendo
#   i primi due, un CAP di tre lettere si prende il messaggio sbagliato.
if len(cap) != LUNGHEZZA_CAP:
    print(f" [ERRORE] servono {LUNGHEZZA_CAP} cifre, ne hai scritte {len(cap)}")
elif not cap.isdecimal():
    print(" [ERRORE] solo cifre: niente lettere, spazi o trattini")
elif not cap.startswith(PREFISSO_PROVINCIA):
    print(f" [ERRORE] i CAP della provincia iniziano per {PREFISSO_PROVINCIA}")
else:
    # Nessun errore: si alza la bandiera, e alla prossima valutazione
    # della condizione il ciclo si chiude da solo.
    valido = True

# La bandiera serve DOPO il ciclo: dice da quale delle due porte si è usciti,
# dato buono o tentativi esauriti. Senza, il programma non sa distinguerle.
if valido:
    print(f"[OK] CAP registrato: {cap}")
else:
    print("[!] Troppi tentativi: rivolgersi allo sportello 2")
```

13.3 Validazione preventiva: il perimetro di oggi

Il controllo con `.isdecimal()` è una validazione PREVENTIVA: si guarda il dato prima di usarlo. Funziona, si scrive in una riga, e resta buona pratica fra vent'anni.

Non è però la strada completa. Esiste uno strumento apposito per raccogliere l'errore dopo che è successo, invece di prevenirlo: si chiama `try/except`, e non è materia di queste undici giornate.

Qui ci fermiamo alla validazione preventiva: resta la prima linea di difesa, ed è quella giusta per i dati che arrivano da una persona.

`.isdecimal()` dice "sono tutte cifre", quindi boccia `-3` e `12.50`: per un CAP va benissimo. Per un importo con la virgola serve `.replace()`: in `05.8` ne trovate uno, e nel commento sono scritti i suoi limiti. Leggeteli, sono istruttivi.

14. ERRORI TIPICI DEI CICLI

Tre danno un messaggio e si vedono. Tre girano in silenzio, ed è molto peggio

14.1 E1 — IndentationError: il ciclo senza corpo

```
for numero in range(3):  
print(f"Giro {numero}")
```

```
File "/Users/giulia/corso-python/giorno_05/conta.py", line 2  
    print(f"Giro {numero}")  
    ^
```

IndentationError: expected an indented block after 'for' statement on line 1

Riga 2 non e' rientrata, quindi il for resta senza corpo. Dopo i due punti Python pretende almeno una riga rientrata: quattro spazi, mai tabulazioni.

Il messaggio dice tutto, e dice anche dove: on line 1 è il vostro for rimasto senza corpo, e la riga citata in cima è quella che dovete rientrare. TRABOCCHETTO: copiando dalla chat si portano dietro spazi invisibili, e l'errore arriva anche quando a schermo il rientro sembra giusto.

14.2 E2 — TypeError: scorrere qualcosa che non ha pezzi

```
for numero in 10:  
    print(numero)
```

Traceback (most recent call last):

```
File "/Users/giulia/corso-python/giorno_05/conta.py", line 1, in <module>  
    for numero in 10:  
        ^^
```

TypeError: 'int' object is not iterable

Iterable = "che si puo' scorrere", cioè fatto di pezzi in fila.

Un int e' un valore solo: non ha pezzi, e dopo `in` non ci puo' stare.

L'aggettivo iterable compare nel messaggio e vale la pena impararlo: è il modo in cui Python vi dice "questo non si scorre". TRABOCCHETTO: per fare dieci giri serve range(10). Una stringa invece è iterabile, e infatti for lettera in "SUD": funziona.

14.3 E3 — TypeError: range con la virgola

```
for numero in range(2.5):  
    print(numero)
```

Traceback (most recent call last):

```
File "/Users/giulia/corso-python/giorno_05/conta.py", line 1, in <module>  
    for numero in range(2.5):  
        ^^^^^^^^^^^
```

TypeError: 'float' object cannot be interpreted as an integer

range conta i giri, e mezzo giro non esiste: accetta solo interi.

```
range(totale / 2)    -> TypeError: / restituisce sempre un float  
range(totale // 2)   -> funziona: // restituisce un int
```

TRABOCCHETTO: succede quando il numero arriva da una divisione. La divisione / restituisce sempre un float, anche quando il risultato è tondo: 10 / 2 vale 5.0, non 5.

14.4 E4 — L'off-by-one: un giro in meno, e nessun errore

```
QUANTI_PROMEMORIA = 10
```

```
# TRABOCCHETTO: qui non c'è nessun errore e il programma gira. range(1, 10)
#   si ferma prima di 10, quindi stampa nove promemoria: il decimo paziente
#   non riceve niente. Forma giusta: range(1, QUANTI_PROMEMORIA + 1).
for numero in range(1, QUANTI_PROMEMORIA):
    print(f"Promemoria {numero}")
```

Il modo per accorgersene non è rileggere: è contare le righe di output la prima volta, lanciando con un numero piccolo. Nove righe invece di dieci non si vedono; con QUANTI_PROMEMORIA = 3 si contano a occhio.

14.4 E4 — Le quattro forme di range da avere in tasca

Cosa volete	Cosa scrivete	Cosa NON scrivere
n giri, indice da 0	<code>range(n)</code>	<code>range(0, n - 1)</code>
n giri, numerati da 1	<code>range(1, n + 1)</code>	<code>range(1, n)</code>
dai 5 ai 20 di 5 in 5, compresi	<code>range(5, 21, 5)</code>	<code>range(5, 20, 5)</code>
all'indietro da n a 1	<code>range(n, 0, -1)</code>	<code>range(n, 1, -1)</code>

Quattro righe, e coprono il novanta per cento dei range che scriverete. La colonna di destra non è teorica: è quello che esce dalle dita quando si scrive in fretta.

14.5 E5 — Il contatore azzerato a ogni giro

```
for peso in range(10, 41, 10):  
    # TRABOCCHETTO: l'inizializzazione è dentro il ciclo, quindi riparte a  
    # ogni giro. Il totale vero sarebbe 100 (10 + 20 + 30 + 40): il  
    # programma stampa 40, cioè soltanto l'ultimo peso.  
    totale = 0  
    totale += peso  
  
# Nessun errore e nessun avviso: solo un numero sbagliato dentro un report  
# che qualcuno userà per decidere qualcosa.  
print(f"Totale: {totale} kg")  
  
# Totale: 40 kg
```

È lo stesso difetto del paragrafo 6.3, e torna qui perché è quello che in aula ricompare più spesso, anche a chi l'ha già visto una volta stamattina.

14.6 E6 — La riga uscita dal ciclo per indentazione

```
totale = 0

for peso in range(10, 41, 10):
    print(f"Collo da {peso} kg")
# TRABOCCHETTO: errore gemello del precedente, con l'indentazione al
# contrario. Questa riga è allineata con il for, quindi sta FUORI dal
# ciclo: viene eseguita una volta sola, dopo, con l'ultimo valore rimasto
# nella variabile del ciclo.

totale += peso

print(f"Totale: {totale} kg")

# Totale: 40 kg      (il totale vero e' di nuovo 100)
```

In un for produce un numero sbagliato e non si vede affatto; in un while, se la riga uscita era l'aggiornamento della condizione, produce un ciclo infinito e si vede subito. Delle due, quella che costa di più è la prima.

14.7 Pronto soccorso: i sintomi con un messaggio

Sintomo	Causa probabile	Prima cosa da guardare
IndentationError: expected an indented block	corpo del ciclo mancante	la riga sotto i due punti
TypeError: 'int' object is not iterable	manca range()	dopo la parola in
TypeError: 'float' object cannot be...	range con un decimale	usare // invece di /
ValueError: invalid literal for int()	conversione prima del controllo sentinella	l'ordine delle righe nel while
NameError dopo il ciclo	il ciclo non ha fatto nemmeno un giro	gli estremi di range
ZeroDivisionError sulla media	zero elementi raccolti	if quanti > 0 prima di dividere

Si usa partendo dal sintomo, non dal codice: rileggere trenta righe sperando che l'errore salti fuori costa dieci minuti, la riga giusta ne costa uno.

14.7 Pronto soccorso: i sintomi senza nessun messaggio

Sintomo	Causa probabile	Prima cosa da guardare
Un giro in meno o in più	off-by-one	l'estremo destro di range
Il totale è l'ultimo valore	inizializzazione dentro il ciclo	il rientro della riga totale = 0
Righe uguali all'infinito	la condizione non cambia mai	l'aggiornamento dentro il while
Il programma continua a chiedere	sentinella non normalizzata	.strip() e .lower()
Nessun output e nessun errore	condizione già falsa in partenza	il valore iniziale della variabile

Questi cinque sono i costosi: il programma gira, non protesta, e restituisce un numero che qualcuno userà per decidere qualcosa. L'ultimo è quello che fa più paura, perché sembra che il programma non sia nemmeno partito.

15. RIPETERE SÌ, CONSERVARE NO: IL PERIMETRO DI OGGI

Oggi non avete imparato a fare cose nuove: avete imparato a farle molte volte

15.1 I programmi dei Giorni 03 e 04, con un ciclo attorno

Programma	Cosa faceva	Cosa diventa oggi
il filtro a punteggio del Giorno 04, cap. 14	un messaggio, un verdetto	05.7: una casella intera, con le statistiche
04.7 — punteggio e soglia al colloquio	un candidato, un esito	05.3: tutti i candidati, con media e massimo
04.2 — sconto a soglie	un carrello, uno sconto	05.8: articoli a ripetizione e sconto sul totale
03.5 — scontrino allineato	una riga stampata	05.8: una riga per articolo, stampata mentre arriva
04.8 — sportello unico	un modulo compilato	05.4: il modulo che non accetta dati sbagliati

Ogni volta è stato detto: "e su cinquecento? Quando avremo i cicli". I cicli sono arrivati: sono quelli di stamattina. Non è codice nuovo, è lo stesso codice dentro una scatola che lo ripete.

15.1 Il tariffario delle parole spia: identico al Giorno 04

Parola spia	Peso	Perché da sola non basta
gratis	3	lo scrive anche una promozione legittima
vinci	3	e lo contiene anche la parola provincia
clicca subito	4	l'indizio più forte, insieme a credenziali
offerta imperdibile	2	commerciale, non per forza spam
urgente	1	lo scrive anche un cliente arrabbiato
credenziali	4	la parola delle truffe vere
rimborso	2	lo scrive chi ha diritto a un rimborso

Soglia: 5 punti. Da 5 in su il messaggio è classificato come spam. I numeri sono quelli del Giorno 04 e non cambiano oggi: quello che cambia è quanti messaggi ci passano dentro.

15.1 Due livelli di vita delle variabili, in 05.7

```
SENTINELLA = "fine"

SOGLIA_SPAM = 5

# Vivono per tutta la sessione: FUORI dal ciclo
esaminati = 0

spam = 0

punteggio_totale = 0

# TRABOCCHETTO: il massimo parte da -1 e non da 0: con 0, in una casella
#   tutta legittima il "messaggio più sospetto" resterebbe vuoto.
punteggio_massimo = -1

messaggio_piu_sospetto = ""

while True:

    messaggio = input("> ").strip()

    if messaggio.lower() == SENTINELLA:

        break

    # Nascono e muoiono a ogni messaggio: DENTRO il ciclo
    normalizzato = messaggio.lower()

    punteggio = 0

    if "gratis" in normalizzato:

        punteggio += PESO_GRATIS

    # ... gli altri sei if, identici al Giorno 04
```

15.1 La casella intera, con le statistiche

Messaggio 1

```
> Le confermo l'appuntamento in ambulatorio
  [--] nessuna parola spia
  punteggio 0 -> LEGITTIMO
```

Messaggio 2

```
> Urgente: pratica di rimborso in scadenza
  [!] 'urgente' (+1)
  [!] 'rimborso' (+2)
  punteggio 3 -> LEGITTIMO
```

Messaggio 3

```
> Vinci un buono gratis, clicca subito
  [!] 'gratis' (+3)
  [!] 'vinci' (+3)
  [!] 'clicca subito' (+4)
  punteggio 10 -> SPAM
```

Messaggio 4

```
> fine
```

STATISTICHE DELLA CASELLA

Messaggi esaminati 3	Quota di spam 33.3%
Messaggi SPAM 1	Punteggio medio 4.33
Messaggi LEGITTIMI 2	

Nel Giorno 04 questo programma girava su un messaggio solo. Oggi ne ha esaminati quanti gliene avete dati, senza che nessuno lo rilanciasse: su cinquecento è la stessa identica riga di codice.

15.2 Il limite delle variabili scalari: si ripete, non si conserva

Domanda che vi verrà stasera	Risposta
"Come tengo tutti i punteggi per ordinarli?"	Le liste, quando arriveranno le strutture dati
"Come conto quante volte compare ogni parola?"	I dizionari, sempre con le strutture dati
"Come evito di riscrivere il classificatore in tre programmi?"	Le funzioni, domani
"Come leggo i messaggi da un file invece che a mano?"	I file, in chiusura di queste giornate
"E se l'utente scrive venti invece di 20?"	try/except: esiste, ma non è materia di queste giornate

In 05.8 la cassa stampa la riga di ogni articolo mentre arriva, perché non c'è nessun posto dove metterla da parte. L'unica memoria di un ciclo, con quello che sapete, sono le variabili scalari: un totale, un contatore, un massimo. Non è un elenco di rinvii: è la mappa di quello che viene dopo. L'ultima riga però è un confine, non un rinvio.

15.3 Sei righe attorno al codice che c'era già

Giulia non ha copiato niente cinquecento volte:

ha aggiunto sei righe attorno a quelle che aveva già.

Il responsabile ha guardato i tre numeri usciti e ha chiesto

la cosa che chiedono sempre: "e per la settimana scorsa?".

Quella risposta arriva quando i messaggi verranno da un file,

e non manca molto.

Per chi vuole andare oltre: esiste una funzione che, mentre scorre una

collezione, vi dà anche il numero del giro — si chiama `enumerate()`, e la

incontrerete con le liste, perché è lì che serve davvero. Oggi il numero

del giro ce lo contiamo a mano, e non è un male.

RIEPILOGO DEL GIORNO

Quindici capitoli in due schermate: quello che resta scritto sul quaderno

Riepilogo del giorno (1/2)

Il ciclo si scrive una volta e si ripete: è l'unica risposta a "e su cinquecento?"

range(n) parte da zero. range(a, b) esclude b. I giri sono sempre $b - a$.

Per arrivare a n compreso: range(1, n + 1). Quel + 1 vi accompagnerà per sempre.

La variabile del ciclo conserva l'ultimo valore, e non nasce se il ciclo non gira.

for su stringa: l'unica collezione di oggi, carattere per carattere, senza indici.

Contatore, accumulatore, massimo, minimo, bandiera, testo: si inizializzano PRIMA del ciclo, si aggiornano dentro, si leggono dopo.

L'errore numero uno: l'inizializzazione dentro il ciclo. Nessun errore, dato falso.

while: tre obblighi — inizializzare, condizionare, aggiornare. La condizione si controlla prima di ogni giro, compreso il primo.

Riepilogo del giorno (2/2)

Ciclo infinito: la condizione non cambia mai. Si esce con Ctrl+C, non chiudendo la finestra e non riavviando l'editor.

Sentinella: parola convenuta, mai un dato valido. `.strip()` e `.lower()` prima di confrontare, e si controlla la sentinella PRIMA di convertire.

`while True` più `break` è la forma standard. `continue` salta al giro dopo.

Se sapete quante volte, `for`. Se dipende da quello che succede, `while`.

Annidati: i giri si moltiplicano. Il `print()` che manda a capo sta dentro l'esterno e fuori dall'interno. Nomi diversi ai due livelli, sempre.

Validare: la condizione del `while` è la negazione della regola di validità, con un tetto ai tentativi e una bandiera per sapere come si è usciti.

Grazie.

shaping the skills of tomorrow

